St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026
Class: BE-ITA/B, Semester: VIII
Subject: BlockChain Lab

Experiment – 5

1. **Aim:** To implement smart contract using truffle framework
2. **Objective:** To …
    - explore working of truffle framework.
    - explore working of Ganache local ethernet network.
    - explore deployment scripts.

3. **Lab outcome:** After performing the experiment, the students will be able to **implement** smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)

4. Prerequisite:

- Fundamental knowledge of blockchain
- Knowledge of the Ethereum platform
- Familiarity with the Solidity programming language and JavaScript

5. **Requirements:** The following are the requirements –
Truffle Framework, Ganache Provider, Visual Studio Code etc.

6. Pre-Experiment Theory:

What is Truffle Framework?

We have various setup options for deploying, migrating, and accessing smart contracts. Depending on the level of control and visibility we want into the EVM (Ethereum Virtual Machine), we can choose from using an online IDE like Remix, to running a full Ethereum mining node via Geth. Truffle is a world-class development environment, testing framework and asset pipeline for blockchains using the EVM. Truffle is widely considered the most popular tool for blockchain application development.

Some of the features of Truffle suite are

- Built-in smart contract compilation, linking, deployment and binary management.
- Automated contract testing for rapid development.
- Scriptable, extensible deployment & migrations framework.
- Network management for deploying to any number of public & private networks.
- Package management with EthPM & NPM, using the ERC190 standard.
- Interactive console for direct contract communication.
- Configurable build pipeline with support for tight integration.
- External script runner that executes scripts within a Truffle environment.

7. Laboratory Exercise

A. Steps to be implemented.

To Follow the procedure given below to build smart contract in Remix IDE

1. Download and install Nodejs from command-prompt terminal, if it is not installed.
2. Create a new directory using the command mkdir, with the name Ethereum (or any other name).
3. Change directory to new directory using cd Ethereum
4. Set up truffle using command npm install -g truffle
5. Open folder/directory Ethereum using Visual Studio, observe empty folder.

6. From terminal type command, truffle init and observe the directory Ethereum with new subfolders
7. In Visual Studio, under contract folder create a new file HelloWorld.sol and write a smart contract (as done in exp-1).
8. Under migration folder create a new js file with name 2_helloWorld_migration.js and paste the following js code in the file

```
        var HelloWorld =
   artifacts.require("./HelloWorld.sol"); module.exports =
   function(deployer) {
        deployer.deploy(HelloWorld);
        };
```

9. Open truffle-config.js, delete all its contents and paste the following

```
code module.exports = {
   networks: {
  development: {
   // from: "", // Defaults to first address from Ganache host:
    "127.0.0.1",
   port: 7545, network_id:
    "*"
   }
   }
   };
```

10. Now from terminal run command truffle compile
11. Open Ganache provider and from terminal run command truffle migrate
12. Now to see the output of smart contract run command truffle console

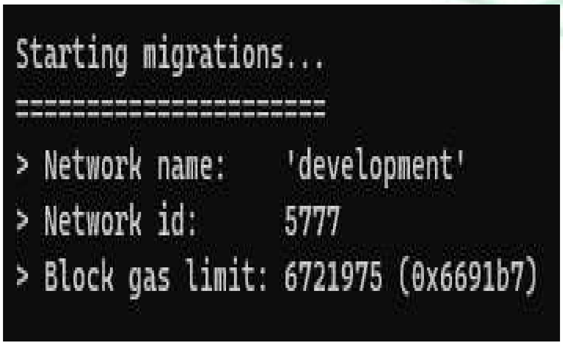
G. Program Code

1. Write a smart contract HelloWorld.sol and deploy it using Ganache and Truffle.

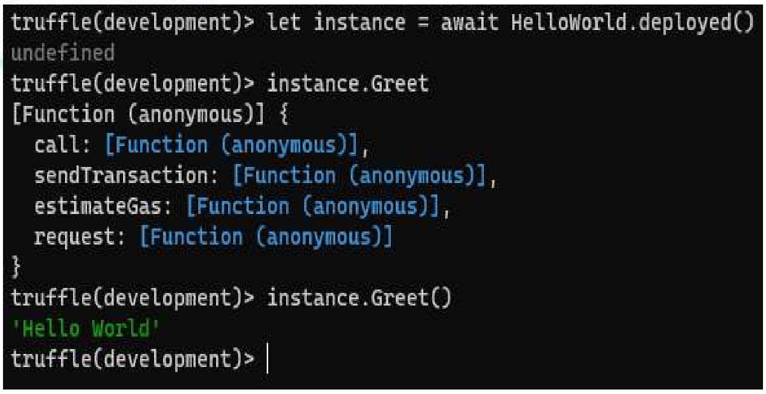
CODE:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.2;

contract HelloWorld {
   function Greet() public pure returns (string memory) {
       return "Hello World";
   }
}
```

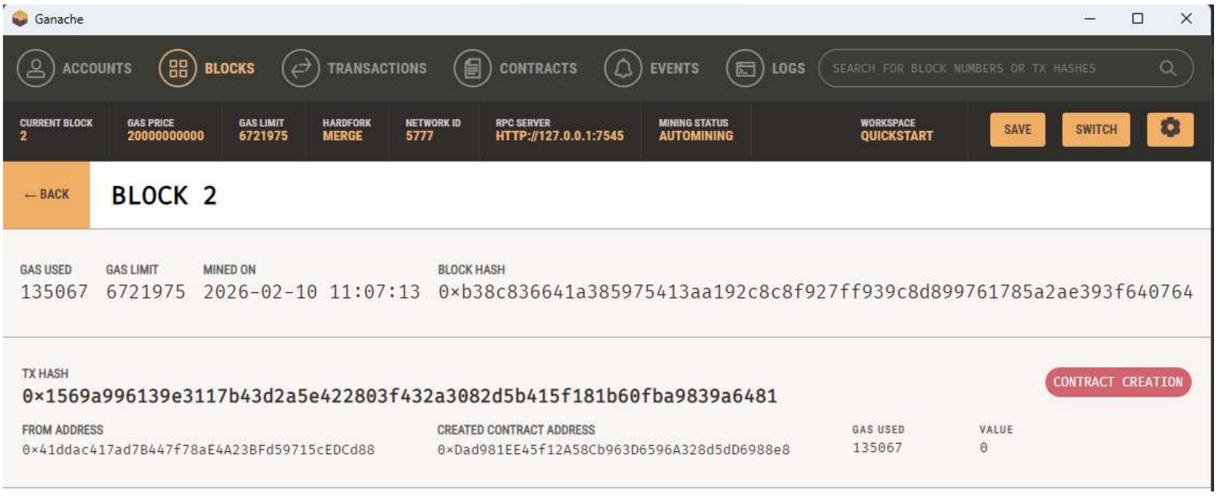
OUTPUT:



```
Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)
```



```
truffle(development)> let instance = await HelloWorld.deployed()
undefined
truffle(development)> instance.Greet
[Function (anonymous)] {
  call: [Function (anonymous)],
  sendTransaction: [Function (anonymous)],
  estimateGas: [Function (anonymous)],
  request: [Function (anonymous)]
}
truffle(development)> instance.Greet()
'Hello World'
truffle(development)> |
```

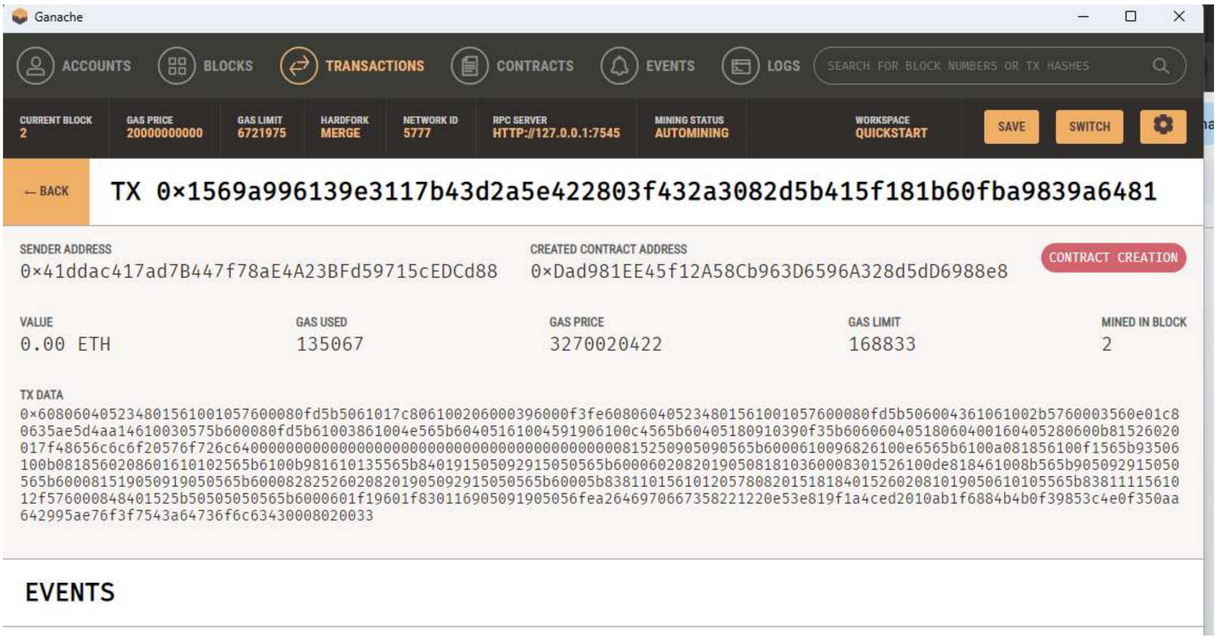


CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART

BLOCK 2

GAS USED 135067
GAS LIMIT 6721975
MINED ON 2026-02-10 11:07:13
BLOCK HASH 0xb38c836641a385975413aa192c8c8f927ff939c8d899761785a2ae393f640764

TX HASH
0x1569a996139e3117b43d2a5e422803f432a3082d5b415f181b60fba9839a6481
FROM ADDRESS 0x41ddac417ad7B447f78aE4A23BFd59715cEDCd88
CREATED CONTRACT ADDRESS 0xDad981EE45f12A58Cb963D6596A328d5dD6988e8
GAS USED 135067
VALUE 0



CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART

TX 0x1569a996139e3117b43d2a5e422803f432a3082d5b415f181b60fba9839a6481

SENDER ADDRESS 0x41ddac417ad7B447f78aE4A23BFd59715cEDCd88
CREATED CONTRACT ADDRESS 0xDad981EE45f12A58Cb963D6596A328d5dD6988e8

VALUE 0.00 ETH
GAS USED 135067
GAS PRICE 3270020422
GAS LIMIT 168833
MINED IN BLOCK 2

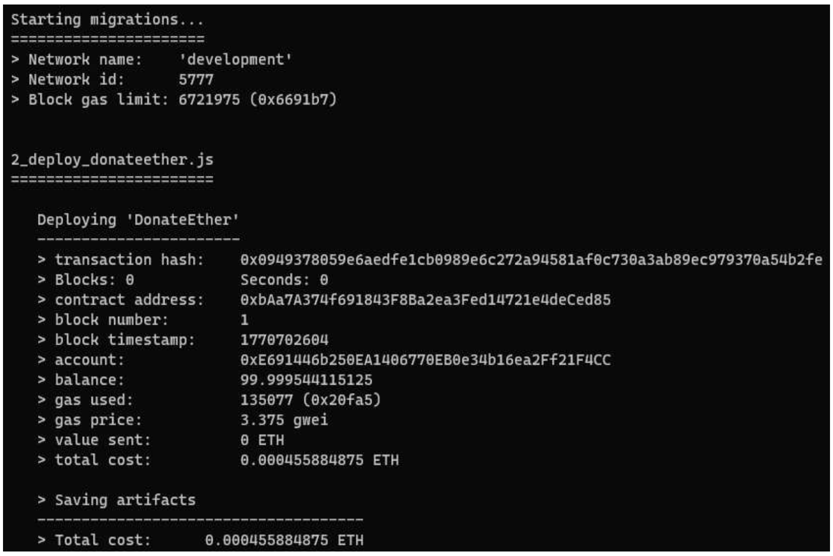
TX DATA

```
0x608060405234801561001057600080fd5b5061017c806100206000396000f3fe608060405234801561001057600080fd5b506004361061002b5760003560e01c8
0635ae5d4aa14610030575b600080fd5b61003861004e565b60405161004591906100c4565b60405180910390f35b60606040518060400160405280600b81526020
017f48656c6c6f20576f726c6400000000000000000000000000000000000000000081525090509056 5b6000610096826100e6565b6100a081856100f1565b93506
100b0818560208601610102565b6100b981610135565b840191505092915050565b600060208201905081810360008301526100de818461008b565b905092915050
565b60008151905091905056 5b600082825260208201905092915050565b60005b83811015610120578082015181840152602081019050610105565b83811115610
12f576000848401525b50505050565b6000601f19601f8301169050919050 56fea2646970667358221220e53e819f1a4ced2010ab1f6884b4b0f39853c4e0f350aa
642995ae76f3f7543a64736f6c63430008020033
```

EVENTS

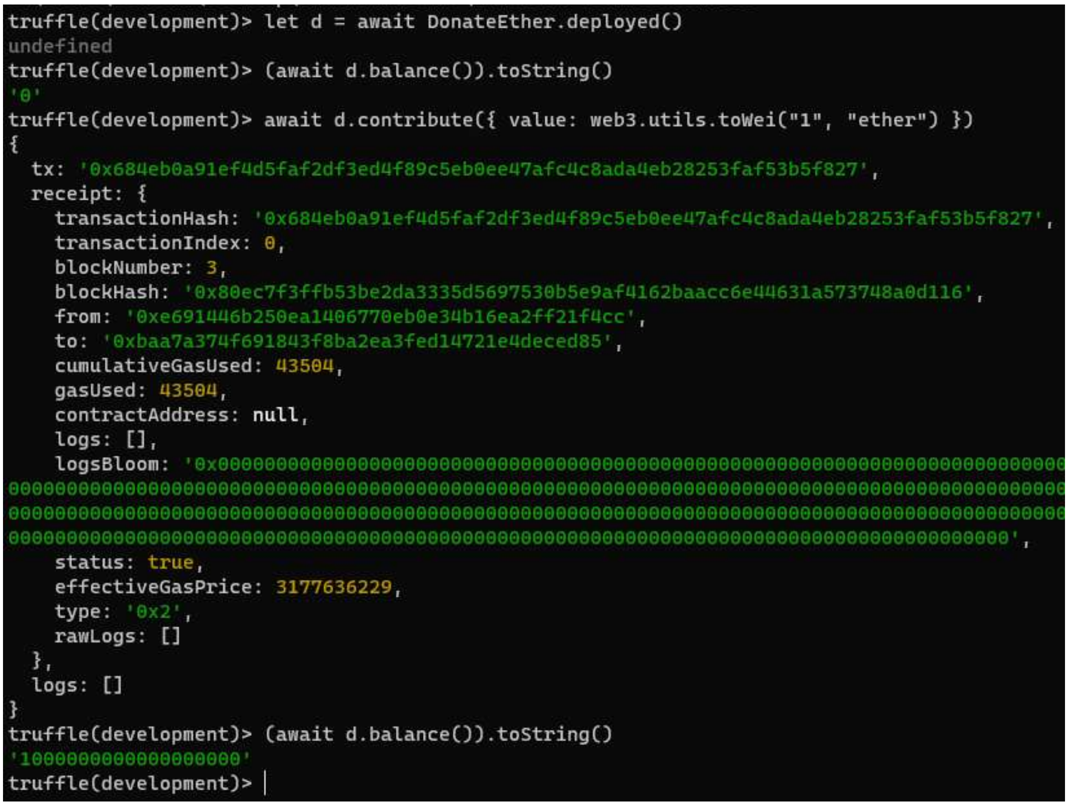
2. Write a program DonateEther.sol, create uint public variable ‘balance’ and initialize it to 0 using constructor() which is public, write a function contribute(), make it public and payable. Inside the function body write balance+=msg.Value; then create migration.js file for this contract as done in HelloWorld program. Deploy the contract on Ganche local Ethereum network using truffle.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.2;

contract DonateEther {
    uint public balance;

    constructor() {
        balance = 0;
    }

    function contribute() public payable {
        balance += msg.value;
    }
}
```

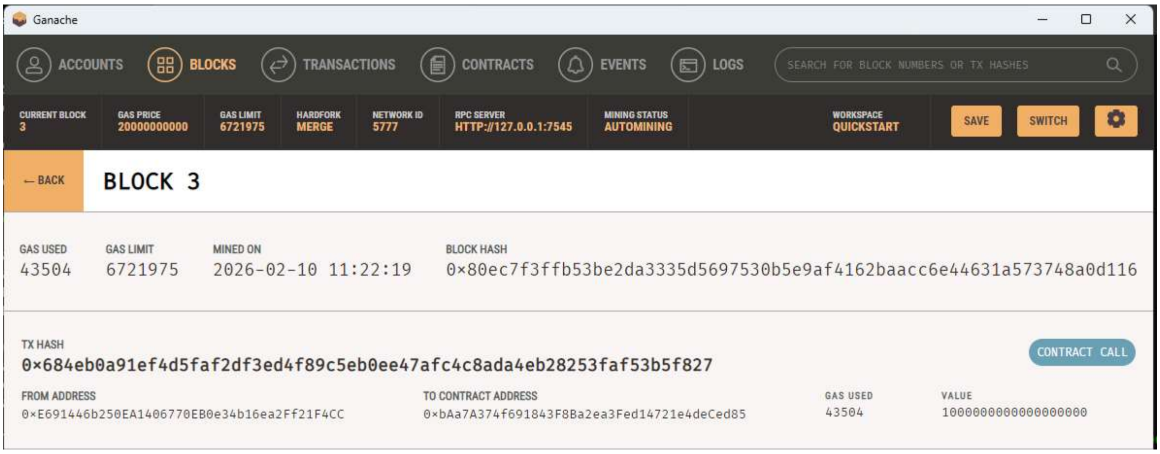


```
Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


2_deploy_donateether.js
=======================

   Deploying 'DonateEther'
   -----------------------
   > transaction hash:    0x0949378059e6aedfe1cb0989e6c272a94581af0c730a3ab89ec979370a54b2fe
   > Blocks: 0            Seconds: 0
   > contract address:    0xbAa7A374f691843F8Ba2ea3Fed14721e4deCed85
   > block number:        1
   > block timestamp:     1770702604
   > account:             0xE691446b250EA1406770EB0e34b16ea2Ff21F4CC
   > balance:             99.999544115125
   > gas used:            135077 (0x20fa5)
   > gas price:           3.375 gwei
   > value sent:          0 ETH
   > total cost:          0.000455884875 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:      0.000455884875 ETH
```

OUTPUT:



```
truffle(development)> let d = await DonateEther.deployed()
undefined
truffle(development)> (await d.balance()).toString()
'0'
truffle(development)> await d.contribute({ value: web3.utils.toWei("1", "ether") })
{
  tx: '0x684eb0a91ef4d5faf2df3ed4f89c5eb0ee47afc4c8ada4eb28253faf53b5f827',
  receipt: {
    transactionHash: '0x684eb0a91ef4d5faf2df3ed4f89c5eb0ee47afc4c8ada4eb28253faf53b5f827',
    transactionIndex: 0,
    blockNumber: 3,
    blockHash: '0x80ec7f3ffb53be2da3335d5697530b5e9af4162baacc6e44631a573748a0d116',
    from: '0xe691446b250ea1406770eb0e34b16ea2ff21f4cc',
    to: '0xbaa7a374f691843f8ba2ea3fed14721e4deced85',
    cumulativeGasUsed: 43504,
    gasUsed: 43504,
    contractAddress: null,
    logs: [],
    logsBloom: '0x0000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000',
    status: true,
    effectiveGasPrice: 3177636229,
    type: '0x2',
    rawLogs: []
  },
  logs: []
}
truffle(development)> (await d.balance()).toString()
'1000000000000000000'
truffle(development)> |
```



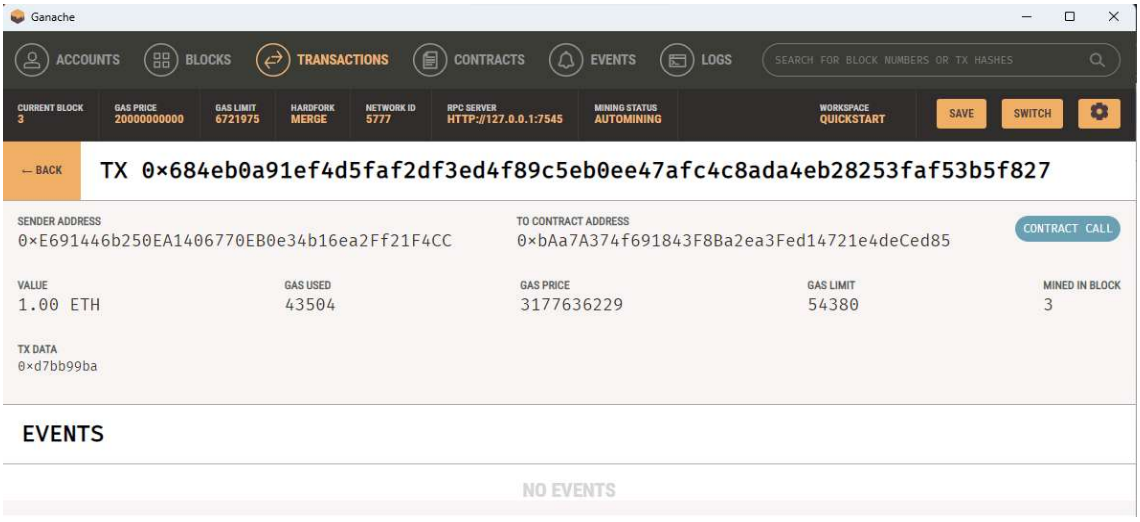
CURRENT BLOCK 3
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART

BLOCK 3

GAS USED 43504
GAS LIMIT 6721975
MINED ON 2026-02-10 11:22:19
BLOCK HASH 0x80ec7f3ffb53be2da3335d5697530b5e9af4162baacc6e44631a573748a0d116

TX HASH
0x684eb0a91ef4d5faf2df3ed4f89c5eb0ee47afc4c8ada4eb28253faf53b5f827

FROM ADDRESS 0xE691446b250EA1406770EB0e34b16ea2Ff21F4CC
TO CONTRACT ADDRESS 0xbAa7A374f691843F8Ba2ea3Fed14721e4deCed85
GAS USED 43504
VALUE 1000000000000000000



CURRENT BLOCK 3
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART

TX 0x684eb0a91ef4d5faf2df3ed4f89c5eb0ee47afc4c8ada4eb28253faf53b5f827

SENDER ADDRESS 0xE691446b250EA1406770EB0e34b16ea2Ff21F4CC
TO CONTRACT ADDRESS 0xbAa7A374f691843F8Ba2ea3Fed14721e4deCed85

VALUE 1.00 ETH
GAS USED 43504
GAS PRICE 3177636229
GAS LIMIT 54380
MINED IN BLOCK 3

TX DATA 0xd7bb99ba

EVENTS

NO EVENTS

8. Post Experimental Exercise-

H. Questions:

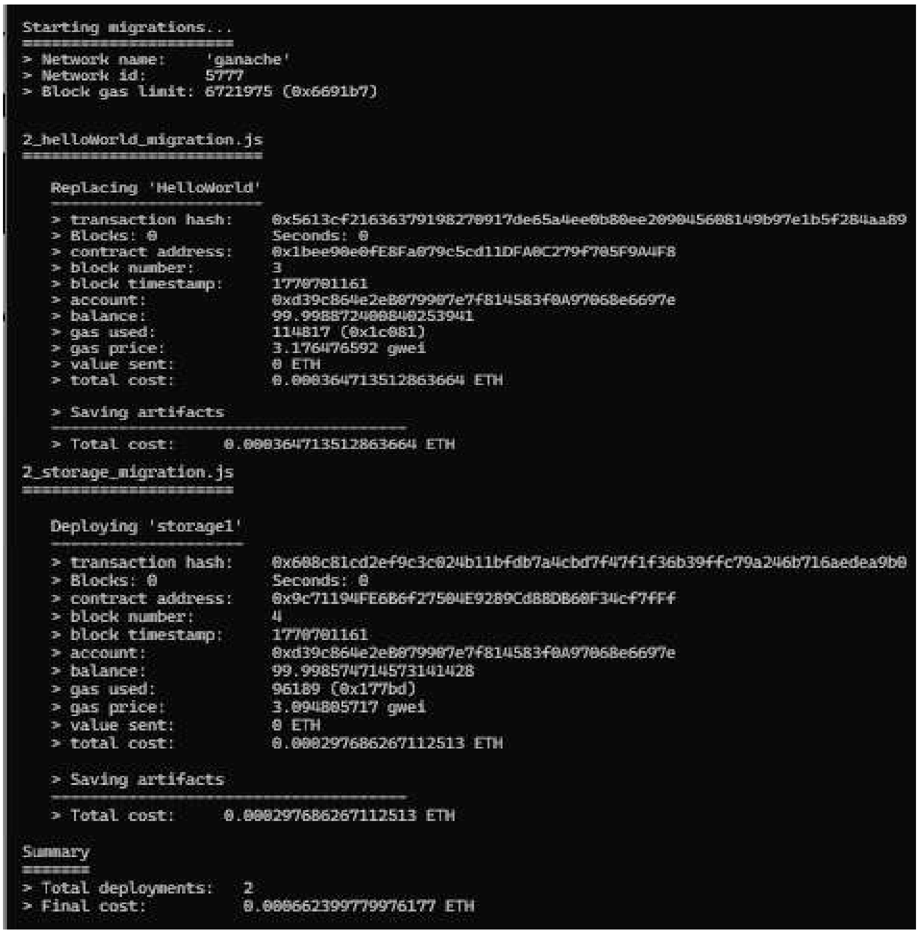
1. List down the details of components of Truffle suite.
2. Deploy all smart contracts studied in expt-1 and expt-2 using Truffle and Ganache.

contracts/storage.sol

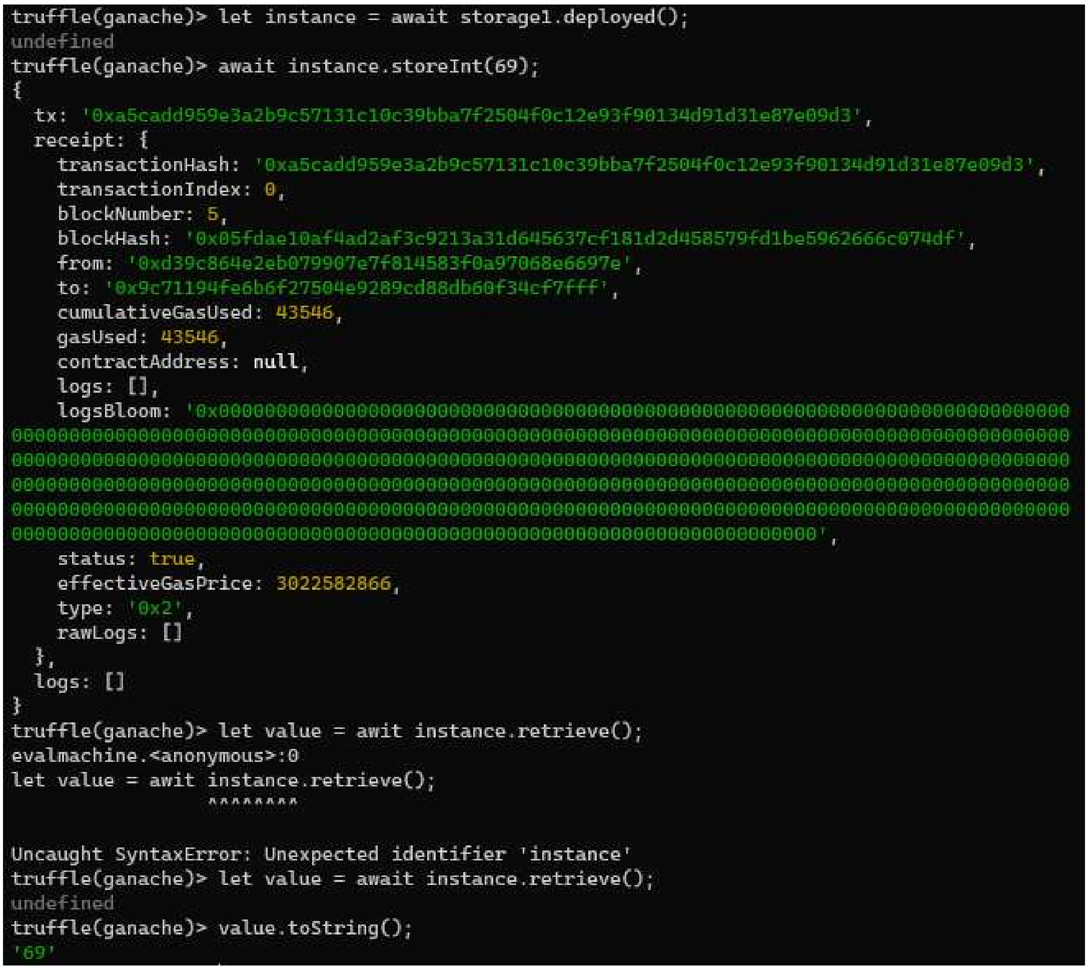
```
pragma solidity ^0.5.16;

contract storage1 {
   uint number;

   function storeInt(uint _num)
public {
      number = _num;
   }

   function retrieve() public view
returns(uint) {
      return number;
   }
}
```

migrations/2_storage_migration.js

```
var storage1 = artifacts.require("storage1");

module.exports = function(deployer) {
   deployer.deploy(storage1);
};
```



```
Starting migrations...
======================
> Network name:    'ganache'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


2_helloWorld_migration.js
=========================

   Replacing 'HelloWorld'
   ----------------------
   > transaction hash:    0x5613cf21636379198270917de65a4ee0b80ee209045608149b97e1b5f284aa89
   > Blocks: 0            Seconds: 0
   > contract address:    0x1bee90e0fE8Fa079c5cd11DFA0C279f705F9A4F8
   > block number:        3
   > block timestamp:     1770701161
   > account:             0xd39c864e2eB079907e7f814583f0A97068e6697e
   > balance:             99.998872400840253941
   > gas used:            114817 (0x1c081)
   > gas price:           3.176476592 gwei
   > value sent:          0 ETH
   > total cost:          0.000364713512863664 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000364713512863664 ETH
2_storage_migration.js
======================

   Deploying 'storage1'
   --------------------
   > transaction hash:    0x608c81cd2ef9c3c024b11bfdb7a4cbd7f47f1f36b39ffc79a246b716aedea9b0
   > Blocks: 0            Seconds: 0
   > contract address:    0x9c71194FE6B6f27504E9289Cd88DB60F34cf7fFf
   > block number:        4
   > block timestamp:     1770701161
   > account:             0xd39c864e2eB079907e7f814583f0A97068e6697e
   > balance:             99.998574714573141428
   > gas used:            96189 (0x177bd)
   > gas price:           3.094805717 gwei
   > value sent:          0 ETH
   > total cost:          0.000297686267112513 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000297686267112513 ETH

Summary
=======
> Total deployments:   2
> Final cost:          0.000662399779976177 ETH
```



```
truffle(ganache)> let instance = await storage1.deployed();
undefined
truffle(ganache)> await instance.storeInt(69);
{
  tx: '0xa5cadd959e3a2b9c57131c10c39bba7f2504f0c12e93f90134d91d31e87e09d3',
  receipt: {
    transactionHash: '0xa5cadd959e3a2b9c57131c10c39bba7f2504f0c12e93f90134d91d31e87e09d3',
    transactionIndex: 0,
    blockNumber: 5,
    blockHash: '0x05fdae10af4ad2af3c9213a31d645637cf181d2d458579fd1be5962666c074df',
    from: '0xd39c864e2eb079907e7f814583f0a97068e6697e',
    to: '0x9c71194fe6b6f27504e9289cd88db60f34cf7fff',
    cumulativeGasUsed: 43546,
    gasUsed: 43546,
    contractAddress: null,
    logs: [],
    logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000',
    status: true,
    effectiveGasPrice: 3022582866,
    type: '0x2',
    rawLogs: []
  },
  logs: []
}
truffle(ganache)> let value = awit instance.retrieve();
evalmachine.<anonymous>:0
let value = awit instance.retrieve();
                 ^^^^^^^^

Uncaught SyntaxError: Unexpected identifier 'instance'
truffle(ganache)> let value = await instance.retrieve();
undefined
truffle(ganache)> value.toString();
'69'
```

I. Results/Observations/Program output:

Present the program input/output results if any and comment on the same.

J. Conclusion:

1. Write what was performed in the experiment.
2. Write which tools you used to perform the experiment
3. Write what you inferred from the output obtained.

K. References:

[1] https://trufflesuite.com/docs/truffle/quickstart/

[2] Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos Dr. Gavin Wood, O'reilly